Tutorial II.II - Package Management

Optimization with Julia



Introduction

Welcome to this beginner-friendly guide on understanding packages and package management in Julia!

Think of packages as pre-written sets of tools that extend what Julia can do. It's like having a toolbox where you can add new tools (packages) to help you solve specific problems. For example, there are packages for working with data, creating visualizations, or solving complex math problems.

The best part? Most Julia packages are free to use, thanks to the open-source community!

> **Note**
> While we'll practice some commands here, you'll typically manage packages in Julia's REPL (Read-Eval-Print Loop), which is like Julia's command center.

Section 1 - Using the Package Manager

Julia's built-in package manager, Pkg, provides a robust set of tools for managing packages. To utilize these tools, start by importing the Pkg module. This module allows you to add, update, and remove packages, and manage environments efficiently.

> **Note**
> Note, `import PackageName` lets you access exported functions with `PackageName.function`, while `using PackageName` imports all exported names into the local namespace for direct access.

Exercise 1.1 - Import the Pkg Module

Import the Pkg module to start managing packages effectively.

```
# YOUR CODE BELOW
```

```
# Test your answer
# Note: This test passes as soon as Pkg is loaded in the current session,
# even if an earlier cell imported it. To really test your own line,
# restart the kernel first.
try
```

```
    Pkg.status()
    println("Pkg module imported successfully!")
catch e
    @error "The Pkg module was not imported yet! Have you used the correct
syntax?"
end
```

Section 2 - Managing Environments and Adding Packages

Think of environments as separate workspaces for different projects. It's like having different folders for different school subjects. It's a best practice to create a new environment for each project to avoid conflicts between package versions. Activate a new environment with `Pkg.activate("new_environment")` and add packages to it with `Pkg.add("PackageName")`. This ensures that each of your projects has a clean, dedicated set of dependencies. Once you add the first package to a new environment, Julia generates two important files:

1. `Project.toml`: Lists direct dependencies of your project.
2. `Manifest.toml`: Contains a complete dependency graph of your project.

These files help ensure reproducibility and should be version controlled with your project. Remember the first tutorial? There, we created the workshop environment in the folder `optimization-workshop` inside your workshop folder, and your tutorial files live in that folder too. To activate it, run:

```
Pkg.activate(".")
```

Here, `"."` stands for the folder Julia runs in. In Zed, that is the folder of your tutorial file, so `"."` points to the workshop's `Project.toml`. In a terminal opened in your workshop folder, use `Pkg.activate("optimization-workshop")` instead. Whenever you start Julia without this environment, activate it before installing new packages. This ensures that the packages are **installed in the environment** and not globally.

> **Caution**
> Check the path carefully: if it does not point to an existing environment, `Pkg.activate` silently creates a new, empty environment at that location, and your packages end up in the wrong place.

> **Tip**
> Our kernel "Optimization with Julia" from the first tutorial already starts in this environment. Therefore, we don't need to activate it most of the time: `Pkg.add` installs packages into the workshop environment directly.

Afterwards, adding packages is straightforward using the `Pkg.add("PackageName")` function. Replace `PackageName` with the actual package name you wish to add.

Exercise 2.1 - Add the DataFrames Package

Let's add a popular package called DataFrames. It's great for working with structured data, like spreadsheets.

```
# YOUR CODE BELOW
```

```
# Test your answer
try
    using DataFrames
    println("Package added successfully!")
catch e
    @error "Package was not added yet! Have you used the correct syntax?"
end
```

Section 3 - Updating and Removing Packages

Just like apps on your phone, packages can be updated or removed when you no longer need them.

- To update a package: `Pkg.update("PackageName")`
- To update all packages: `Pkg.update()`
- To remove a package: `Pkg.rm("PackageName")`

> **Example**
> If you wanted to update DataFrames, you'd use `Pkg.update("DataFrames")`. To remove it, you'd use `Pkg.rm("DataFrames")`.

Note that `Pkg.rm` only removes a package from the current environment. The package itself stays in a shared folder on your computer (called the depot, usually `~/.julia`), so other environments can still use it and adding it again later is fast.

Exercise 3.1 - Which Command?

Imagine you no longer need DataFrames in the current environment. Which function removes it? Assign the name of the function as a string to the variable `remove_command`: `"add"`, `"update"`, or `"rm"`.

```
# YOUR CODE BELOW
remove_command = ""
```

```
# Test your answer
@assert remove_command in ["add", "update", "rm"] "Please assign one of the
strings \"add\", \"update\" or \"rm\" to remove_command."
@assert remove_command == "rm" "Not quite! Have another look at the list above:
which function removes a package from the environment?"
println("Correct! Pkg.rm(\"DataFrames\") removes the package from the current
environment, but it stays in the depot on your computer.")
```

Conclusion

Congratulations! You've completed the tutorial on packages and package management in Julia. These skills are important for effectively managing and utilizing external libraries. Continue to the next file to learn more.

Solutions

You will find the solutions to all exercises online here in the `solutions` folders for each part. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.